

```

        return
    else:
        # RECURSIVE CASE
        # Move to the start position.
        recursiveArgs = []
        turtle.penup()
        turtle.goto(startPosition)
        turtle.setheading(heading)
        recursiveArgs.append({'position':turtle.position(),
                              'heading':turtle.heading()})

        # Erase the middle third.
        turtle.forward(length / 3)
        turtle.pencolor('white')
        turtle.pendown()
        turtle.forward(length / 3)

        # Draw the bump.
        turtle.backward(length / 3)
        turtle.left(60)
        recursiveArgs.append({'position':turtle.position(),
                              'heading':turtle.heading()})
        turtle.pencolor('black')
        turtle.forward(length / 3)
        turtle.right(120)
        recursiveArgs.append({'position':turtle.position(),
                              'heading':turtle.heading()})
        turtle.forward(length / 3)
        turtle.left(60)
        recursiveArgs.append({'position':turtle.position(),
                              'heading':turtle.heading()})

        for i in range(4):
            drawKochCurve(recursiveArgs[i]['position'],
                          recursiveArgs[i]['heading'],
                          length / 3)
        return

def drawKochSnowflake(startPosition, heading, length):
    # A Koch snowflake is three Koch curves in a triangle.

    # Move to the starting position.
    turtle.penup()
    turtle.goto(startPosition)
    turtle.setheading(heading)

```